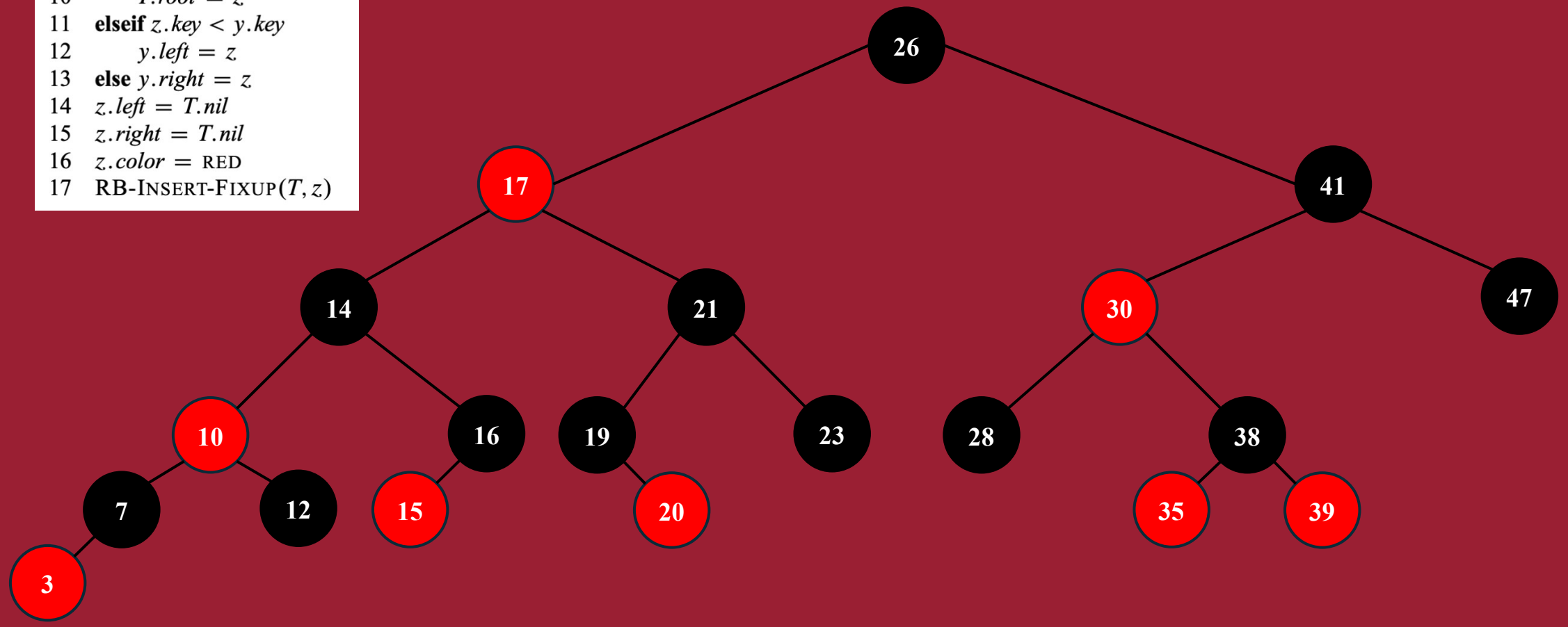
CS 201
Red Black Trees

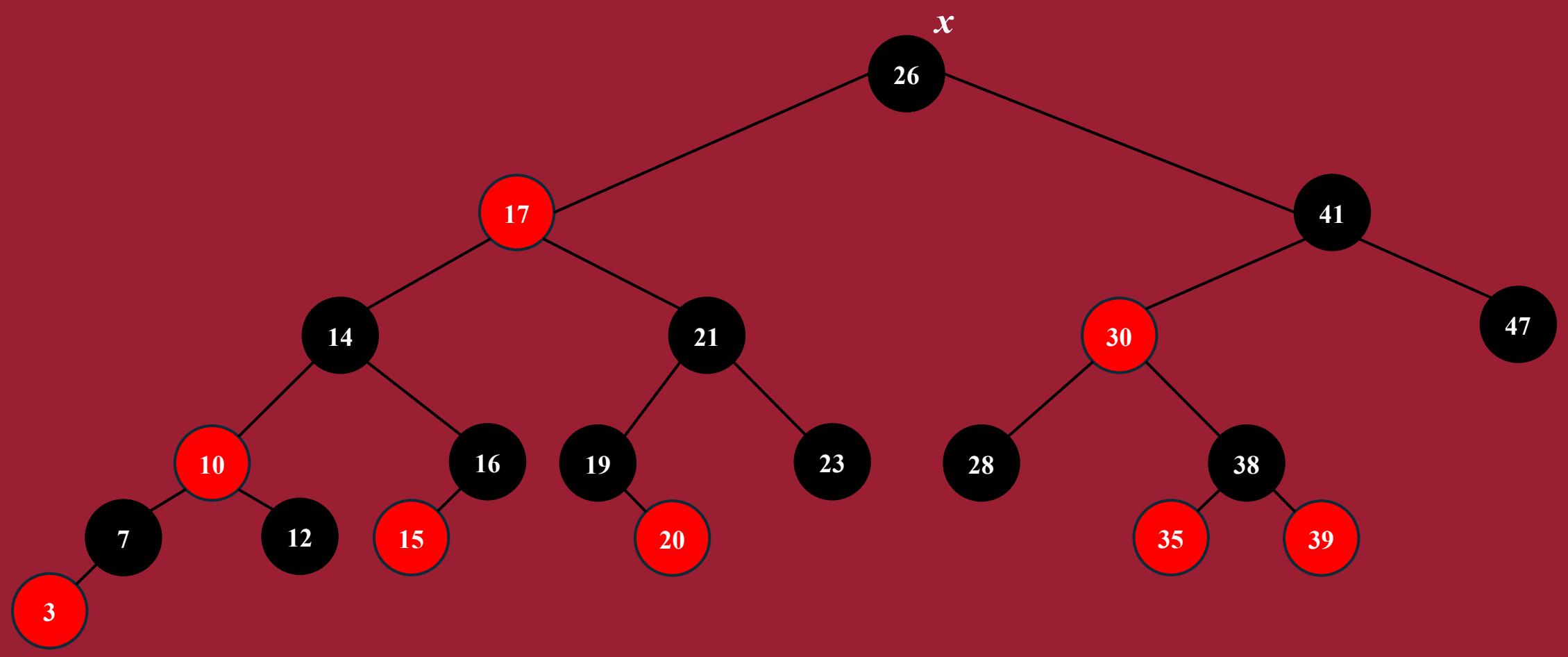
Properties of a Red-Black Tree

1. Every node is either red or black.
2. The root is black.
3. Every leaf (T.nil) is black.
4. If a node is red, then both its children are black.
5. For each node, all simple paths from the node to descendant leaves contain the same number of black nodes.

```
RB-INSERT(T, z)
y = T.nil
x = T.root
while x ≠ T.nil
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
z.p = y
if y == T.nil
    T.root = z
elseif z.key < y.key
    y.left = z
else y.right = z
z.left = T.nil
z.right = T.nil
z.color = RED
RB-INSERT-FIXUP(T, z)
```

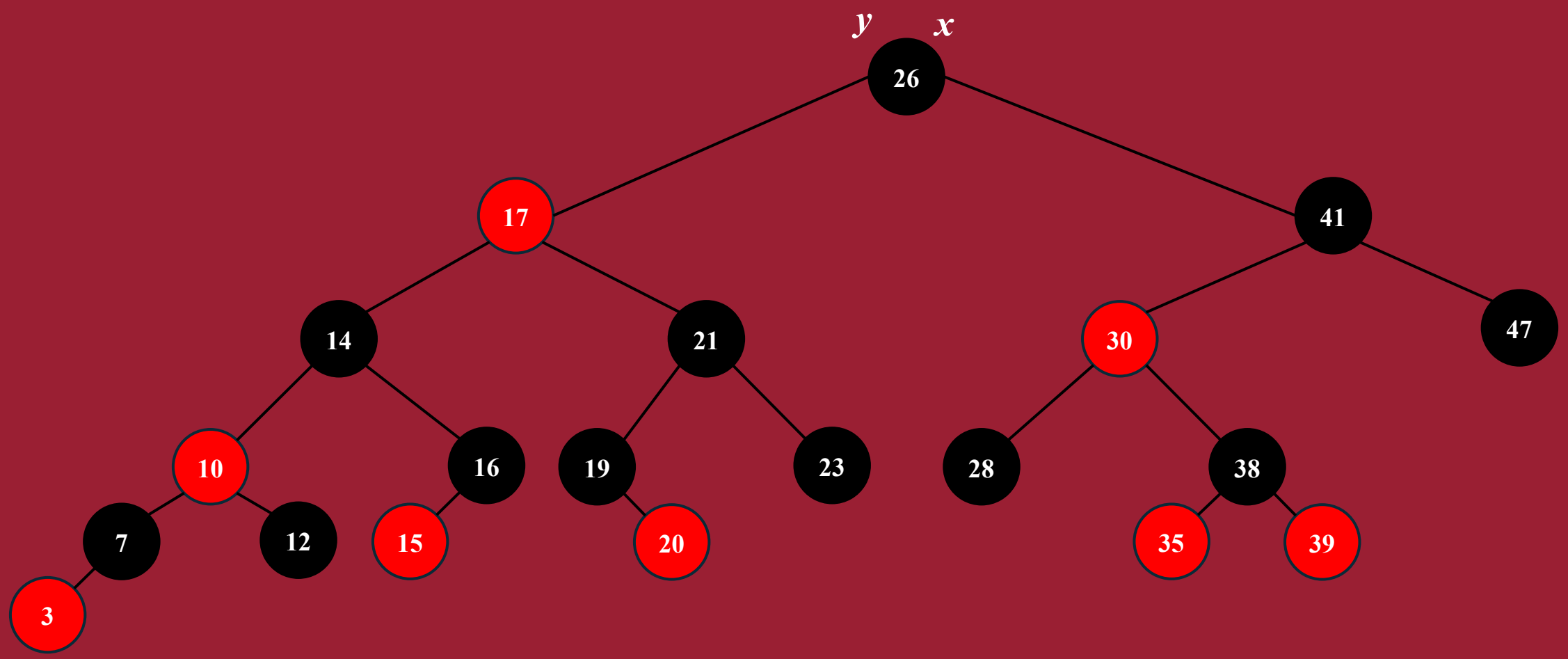


```
y = T.nil
x = T.root
```



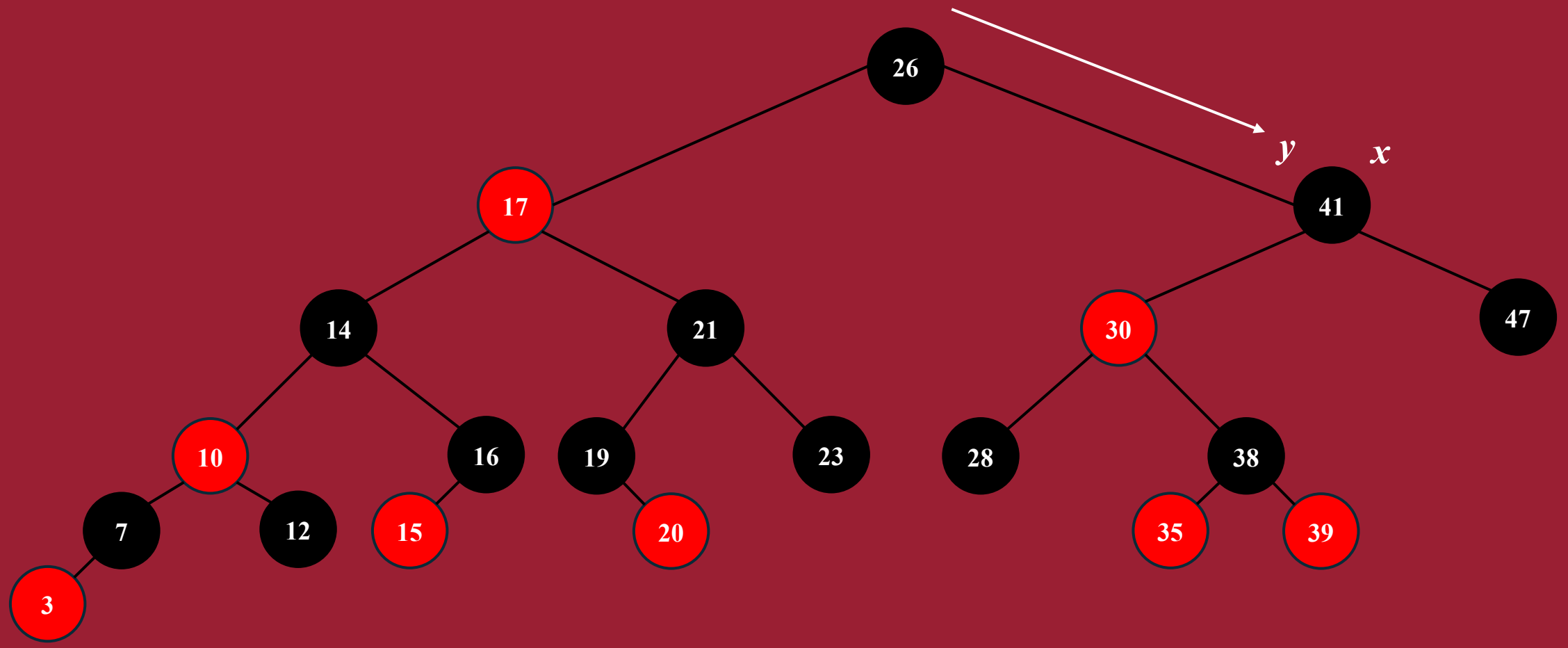
```
while x ≠ T.nil
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
```

36 < 26



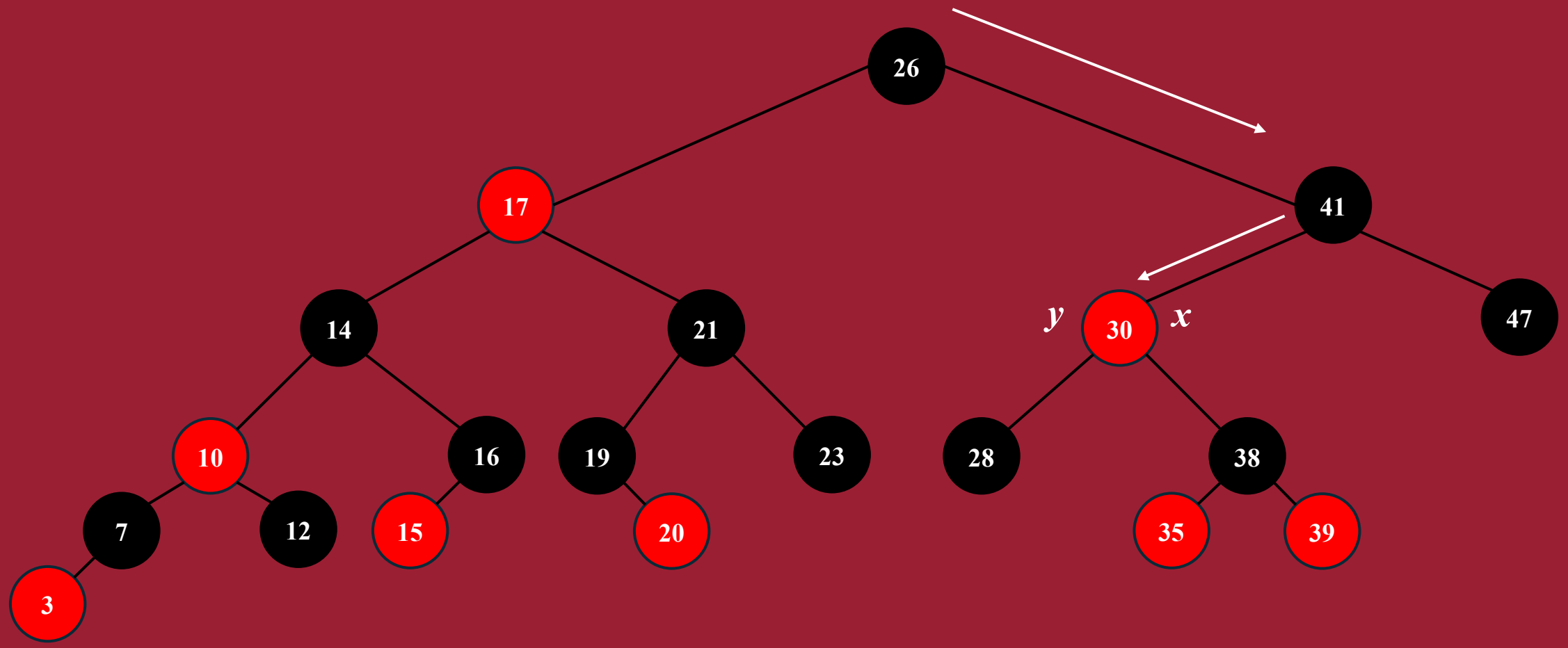
```
while x ≠ T.nil
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
```

36 < 41



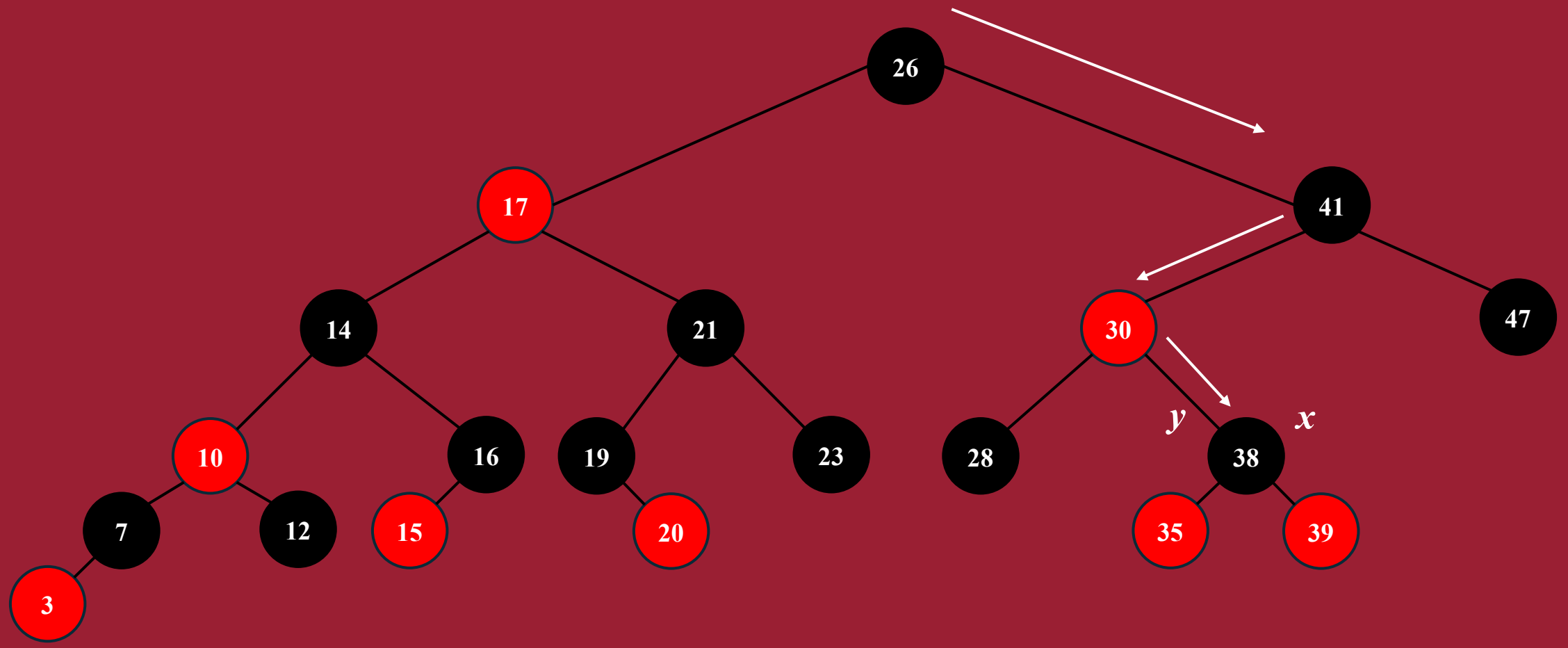
```
while x ≠ T.nil
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
```

36 < 30



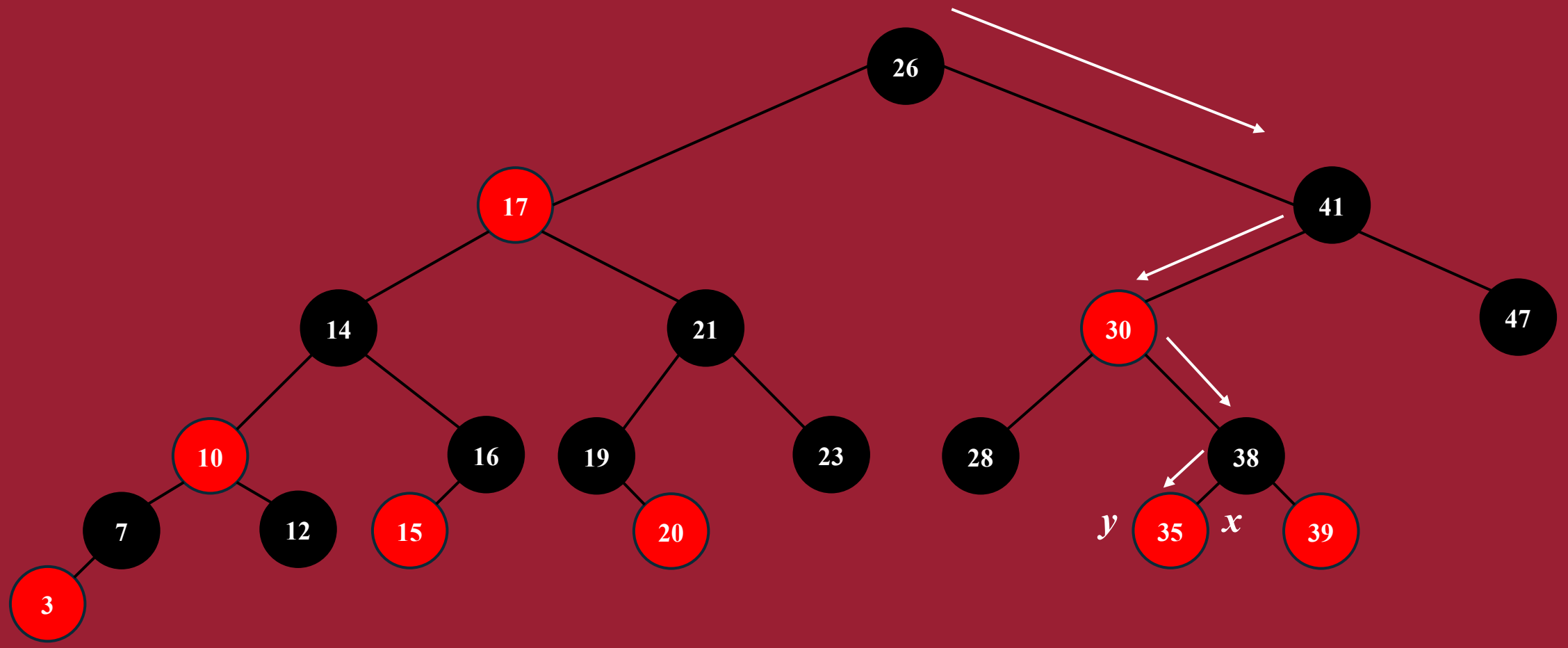
```
while x ≠ T.nil
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
```

36 < 38



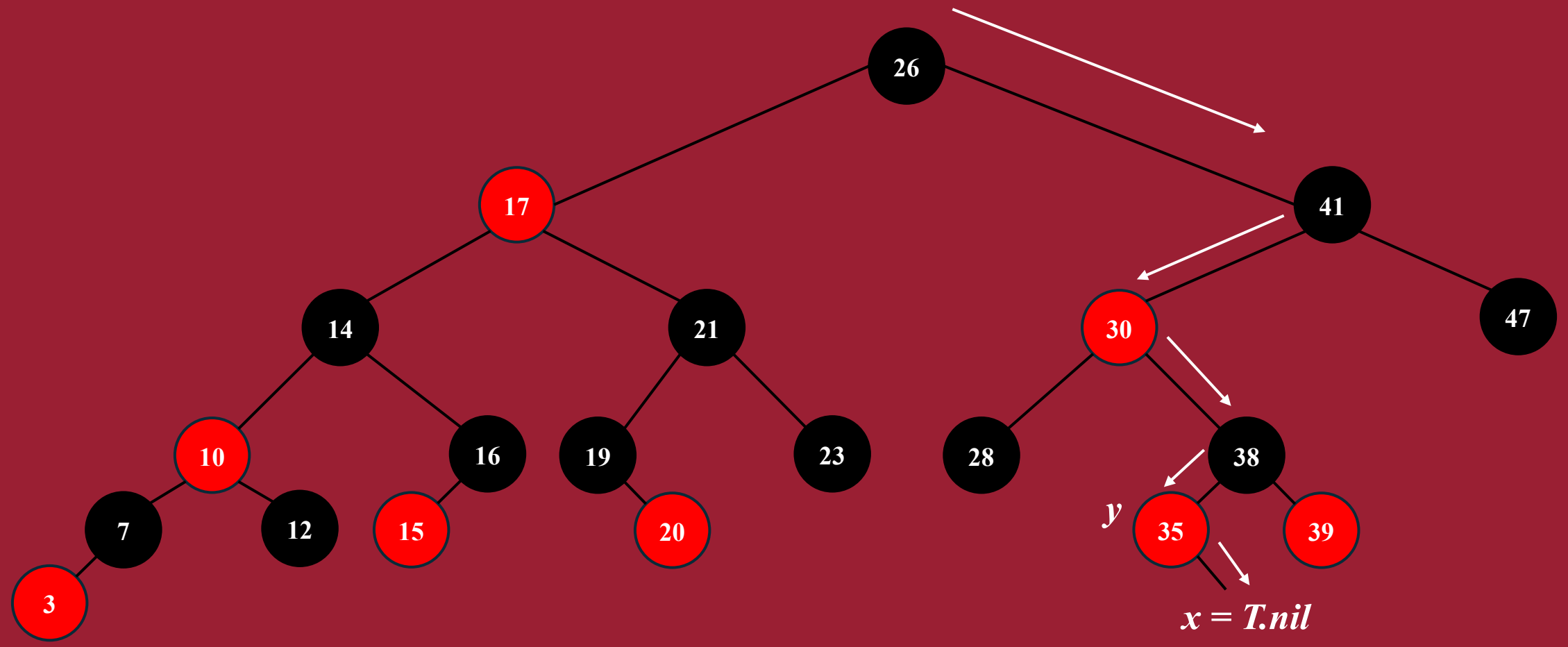
```
while x ≠ T.nil
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
```

36 < 35



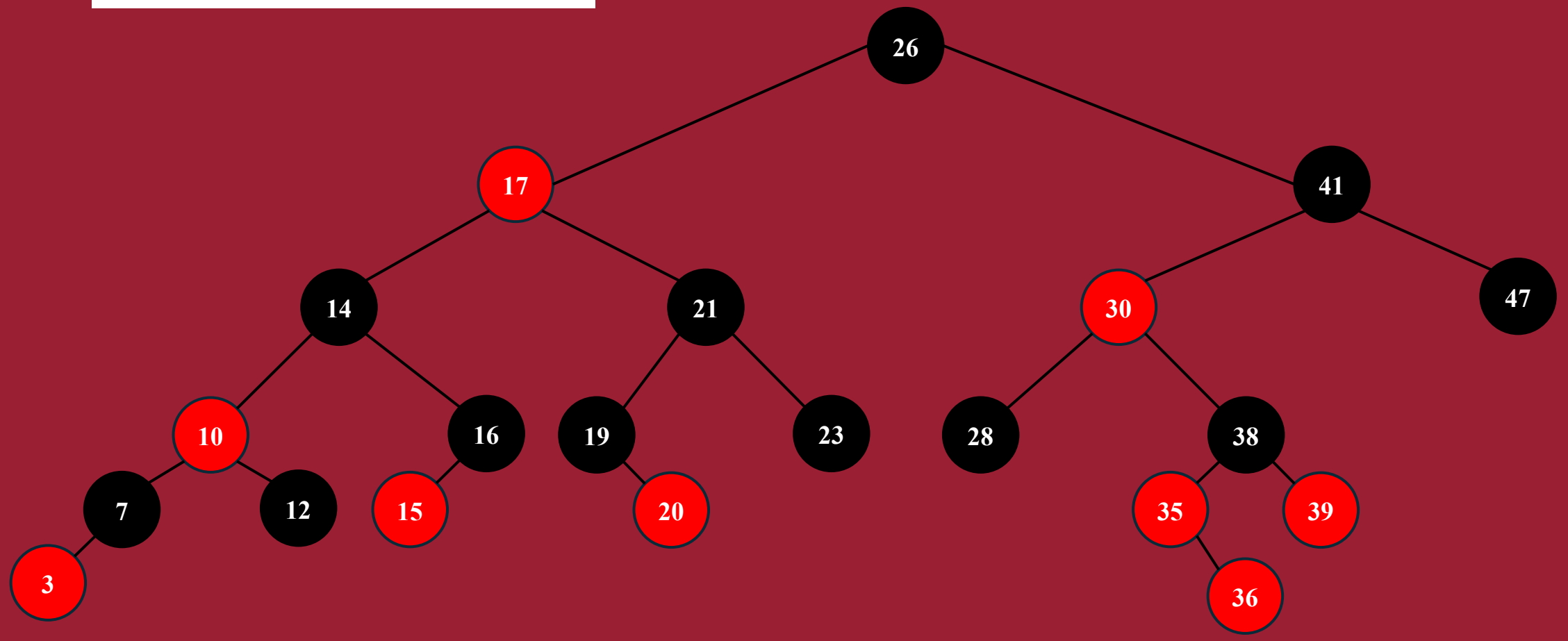
```
z.p = y
if y == T.nil
    T.root = z
elseif z.key < y.key
    y.left = z
else y.right = z
```

$z.key =$ 36 $\geq$ y.right (35)

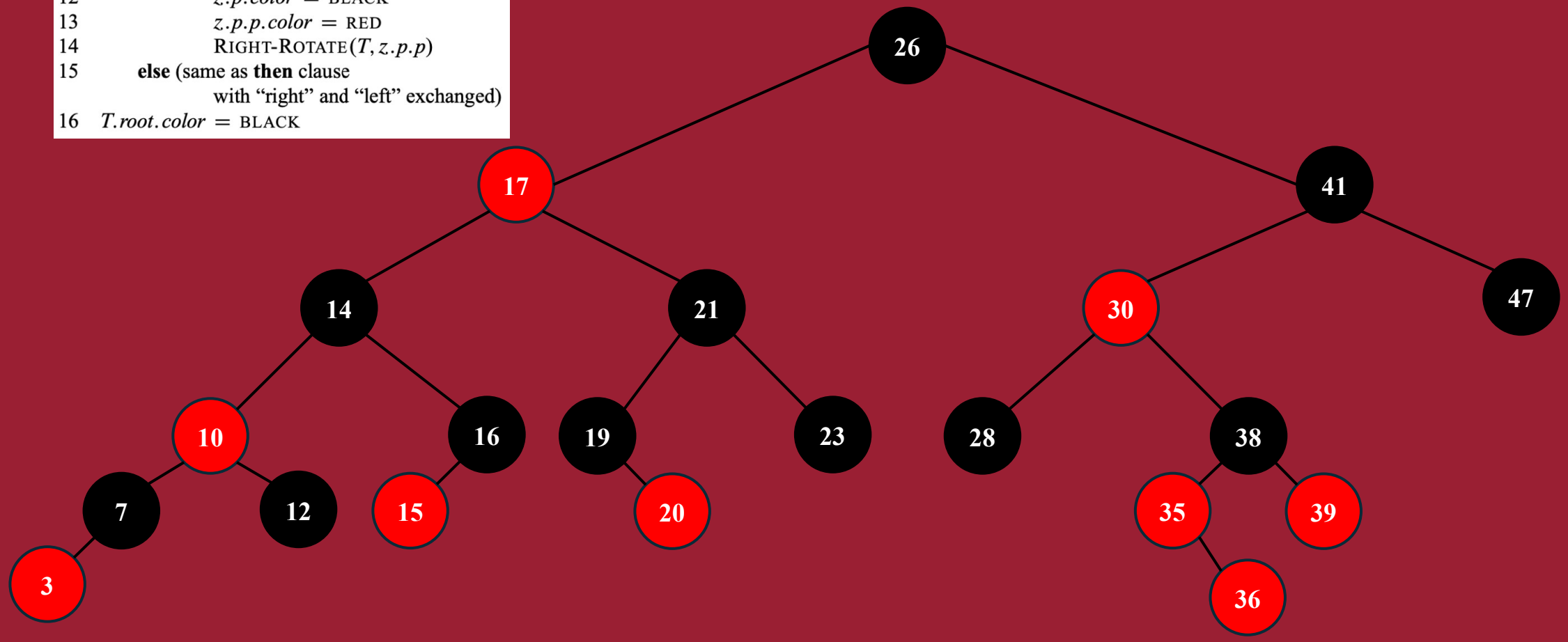


```
z.left = T.nil
z.right = T.nil
z.color = RED
RB-INSERT-FIXUP(T, z)
```

Coloring *z* as red would break rule #4 of the red-black properties (if a node is red, both of its children must be black) if *RB-INSERT-FIXUP(T, z)* is not called.

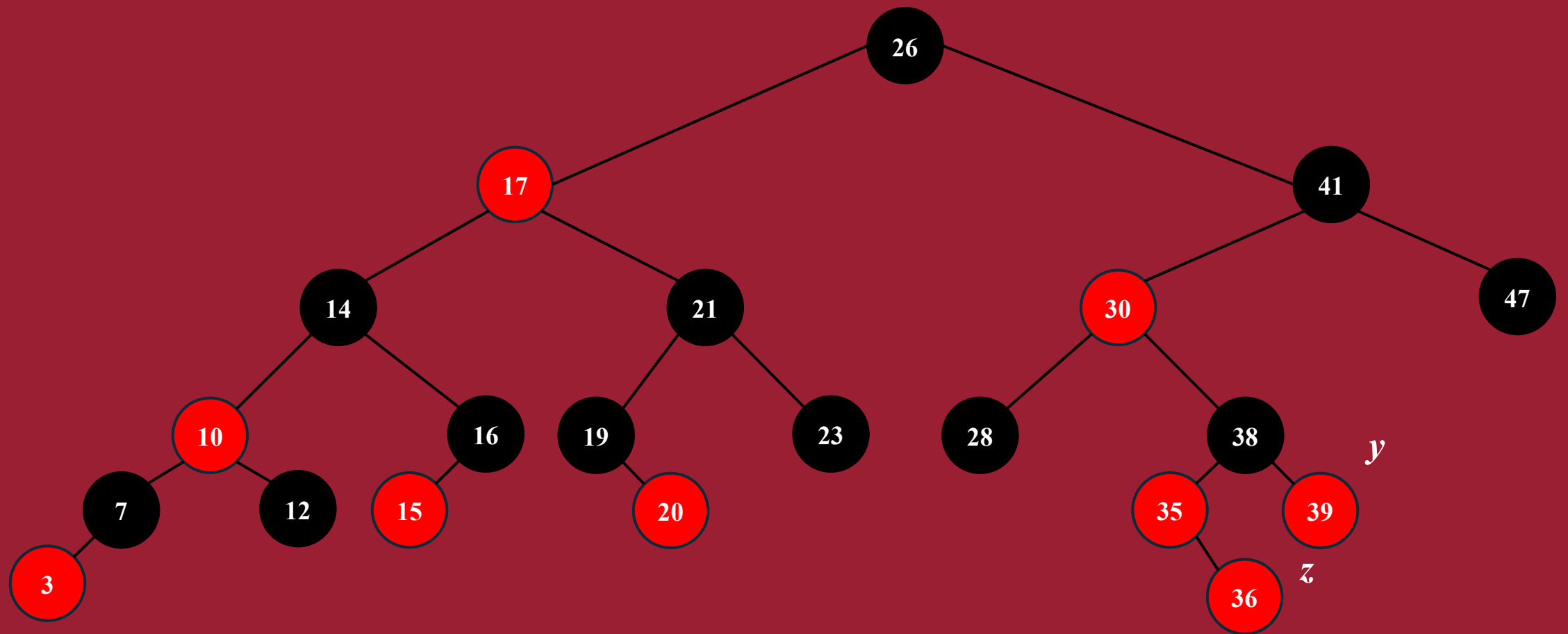


```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p
                LEFT-ROTATE(T, z)
            z.p.color = BLACK
            z.p.p.color = RED
            RIGHT-ROTATE(T, z.p.p)
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```



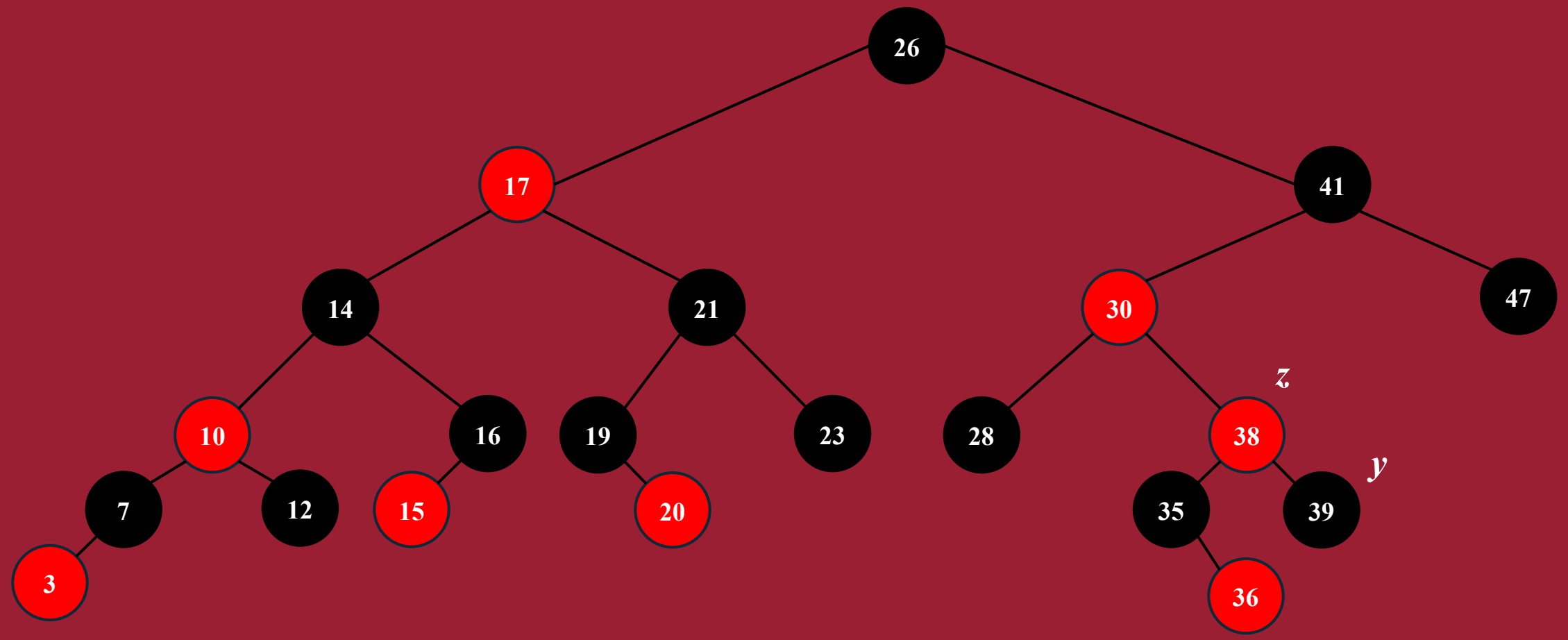
```
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
```

y = z's uncle, and y is red, so **case 1** applies.



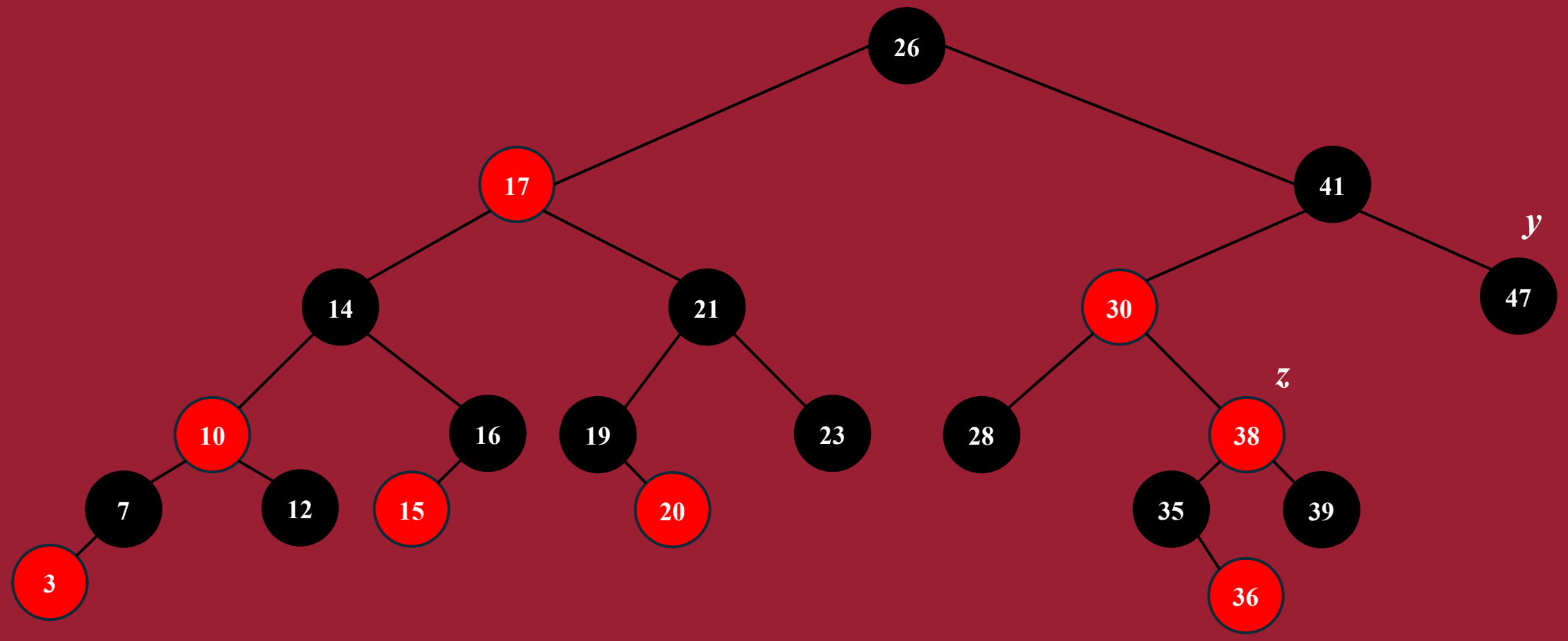
```
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
```

$y = z$'s uncle, and y is red, so **case 1** applies.



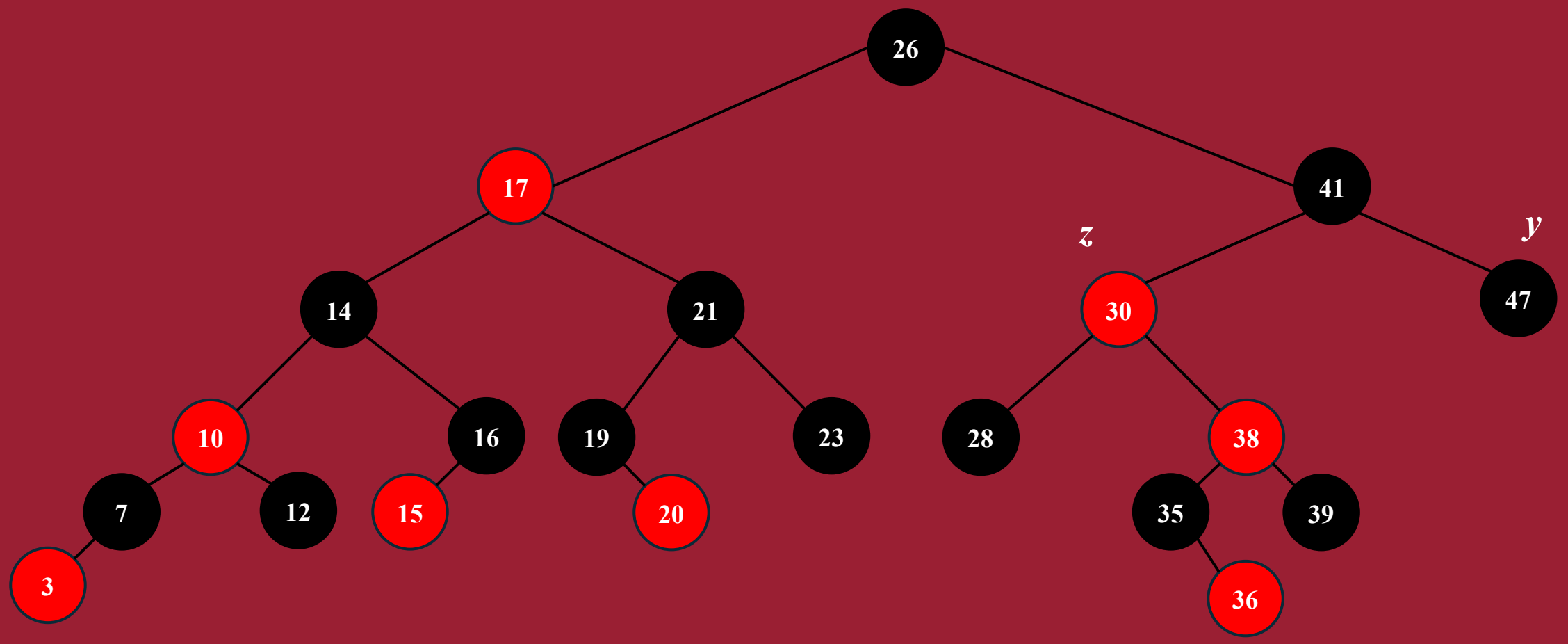
```
else if z == z.p.right
        z = z.p
        LEFT-ROTATE(T, z)
    z.p.color = BLACK
    z.p.p.color = RED
    RIGHT-ROTATE(T, z.p.p)
```

y = z's uncle, but y is now black. z is z's parent's right child. **Case 2** applies.



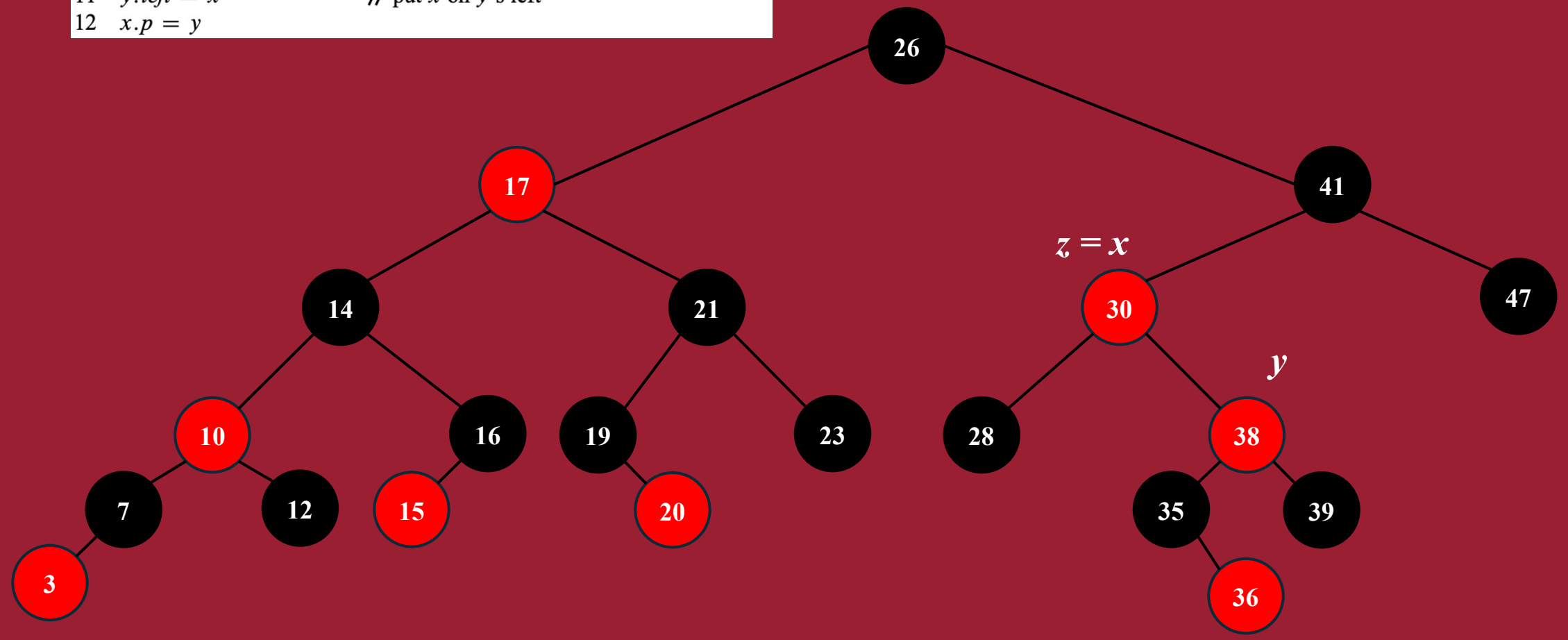
```
else if z == z.p.right
        z = z.p
        LEFT-ROTATE(T, z)
    z.p.color = BLACK
    z.p.p.color = RED
    RIGHT-ROTATE(T, z.p.p)
```

y = z's uncle, but y is now black. z is z's parent's right child. **Case 2** applies.



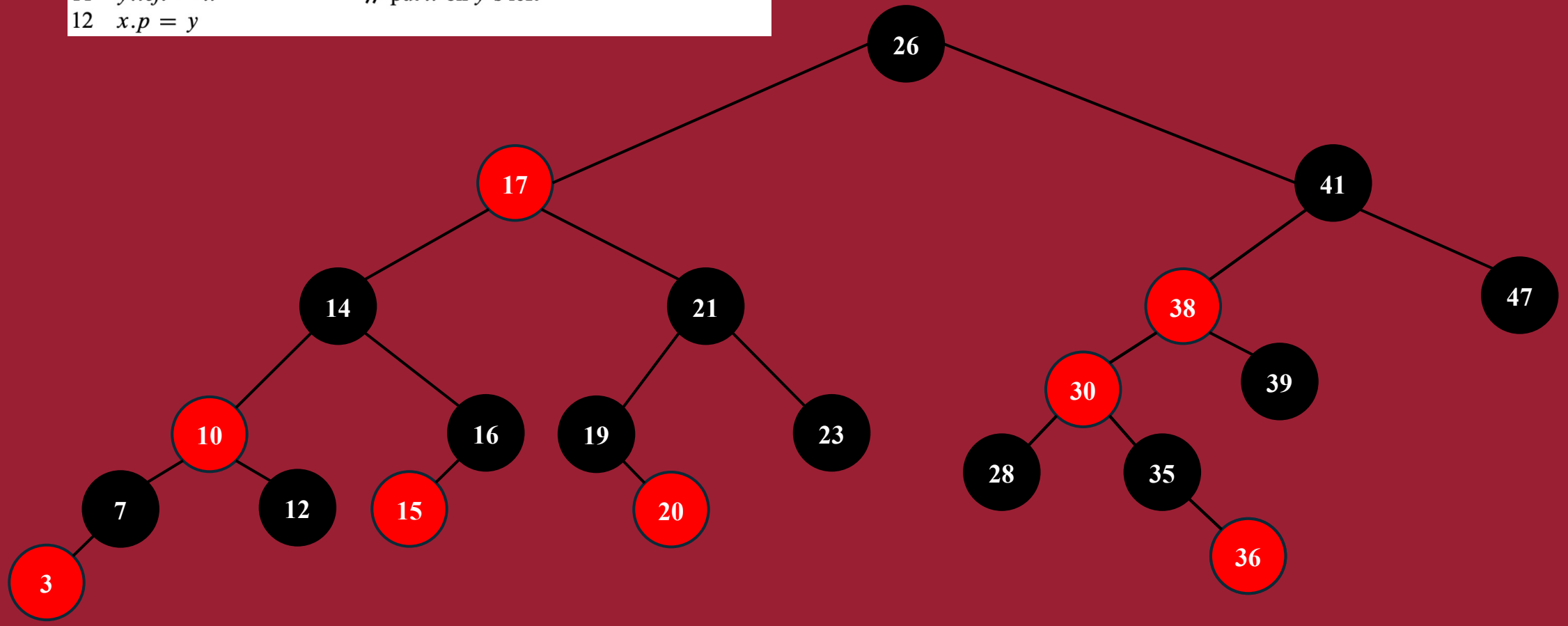
```
LEFT-ROTATE(T, x)
y = x.right                    // set y
x.right = y.left               // turn y's left subtree into x's right subtree
if y.left ≠ T.nil
    y.left.p = x
y.p = x.p                      // link x's parent to y
if x.p == T.nil
    T.root = y
elseif x == x.p.left
    x.p.left = y
else x.p.right = y
y.left = x                     // put x on y's left
x.p = y
```

Call LEFT-ROTATE(T, z). z replaces x in the pseudocode.



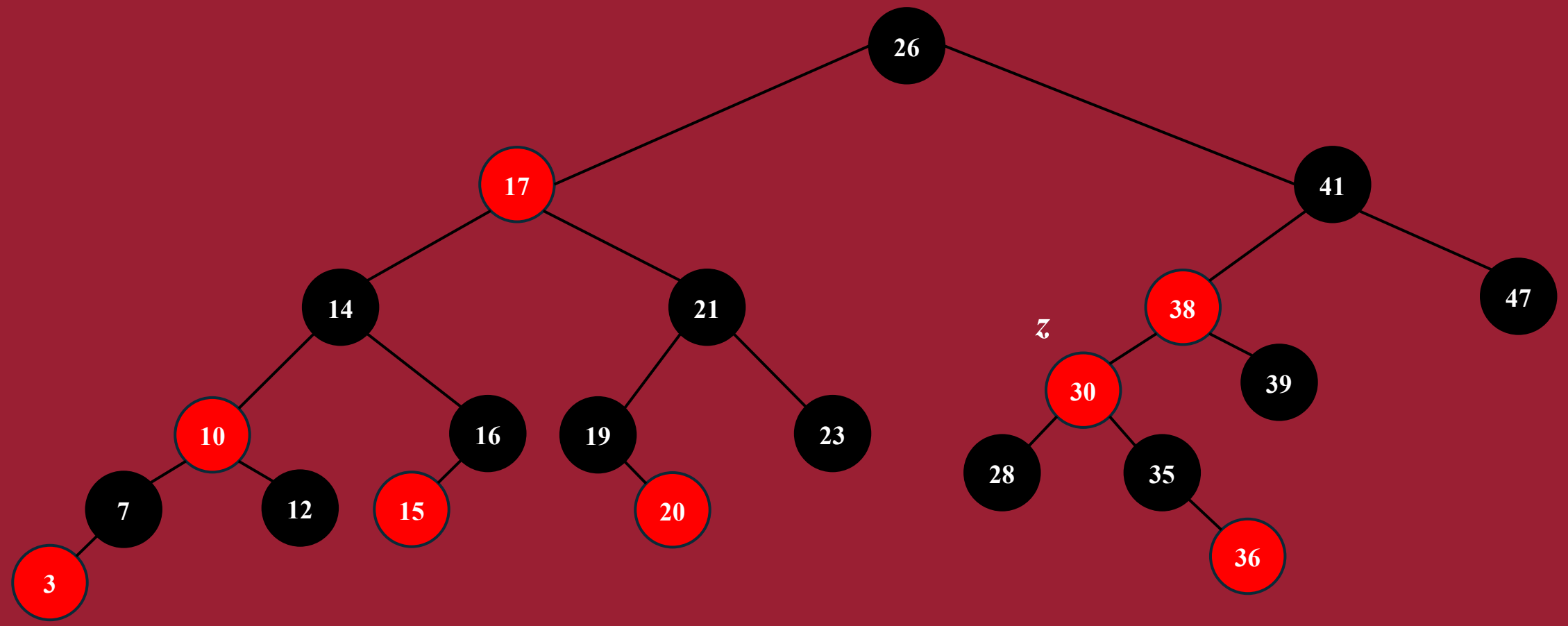
```
LEFT-ROTATE(T, x)
y = x.right                    // set y
x.right = y.left               // turn y's left subtree into x's right subtree
if y.left ≠ T.nil
    y.left.p = x
y.p = x.p                      // link x's parent to y
if x.p == T.nil
    T.root = y
elseif x == x.p.left
    x.p.left = y
else x.p.right = y
y.left = x                     // put x on y's left
x.p = y
```

y = 38;
x.right = y.left = 35;
y.left ≠ T.nil, y.left.p = 38;
y.p = x.p = 41;
x == x.p.left, x.p.left = y = 38;
y.left = x = 30;
x.p = y = 38;



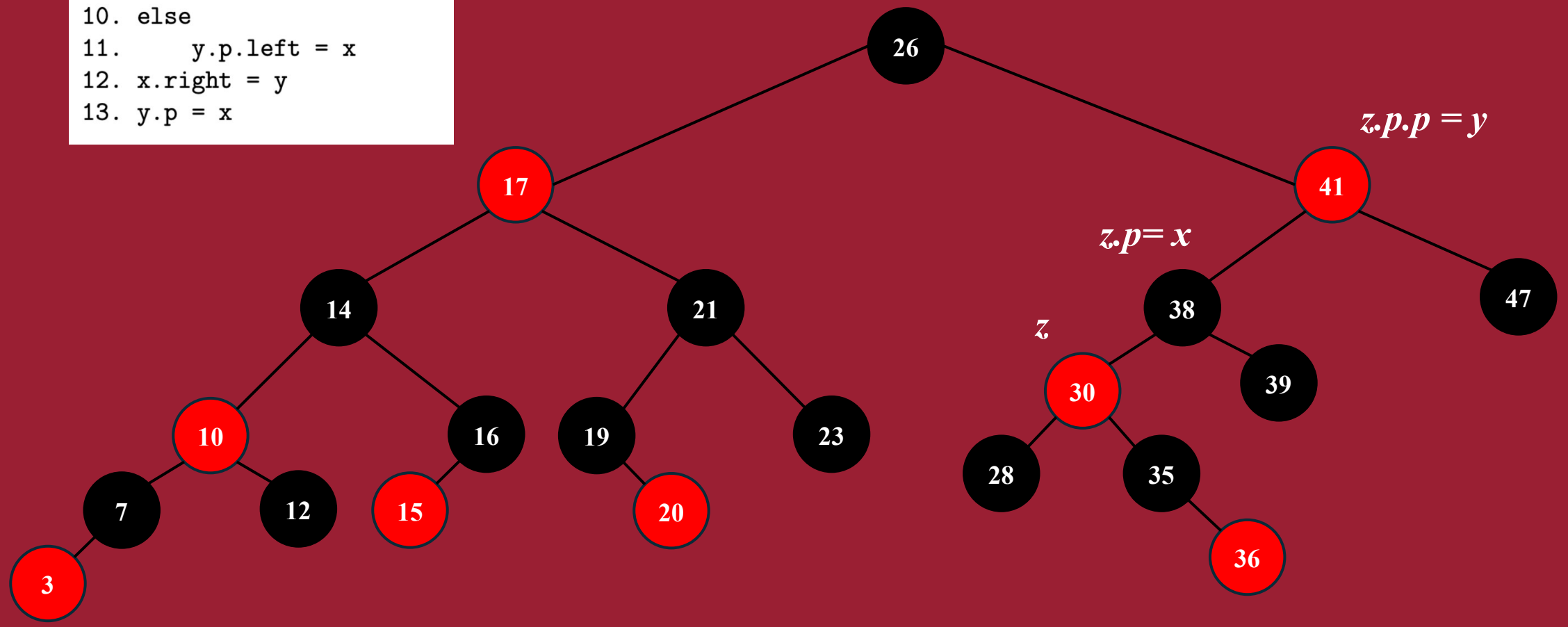
```
else if z == z.p.right
        z = z.p
        LEFT-ROTATE(T, z)
    z.p.color = BLACK
    z.p.p.color = RED
    RIGHT-ROTATE(T, z.p.p)
```

Immediately **case 3** applies. Change colors of z's parent and z's grandparent…



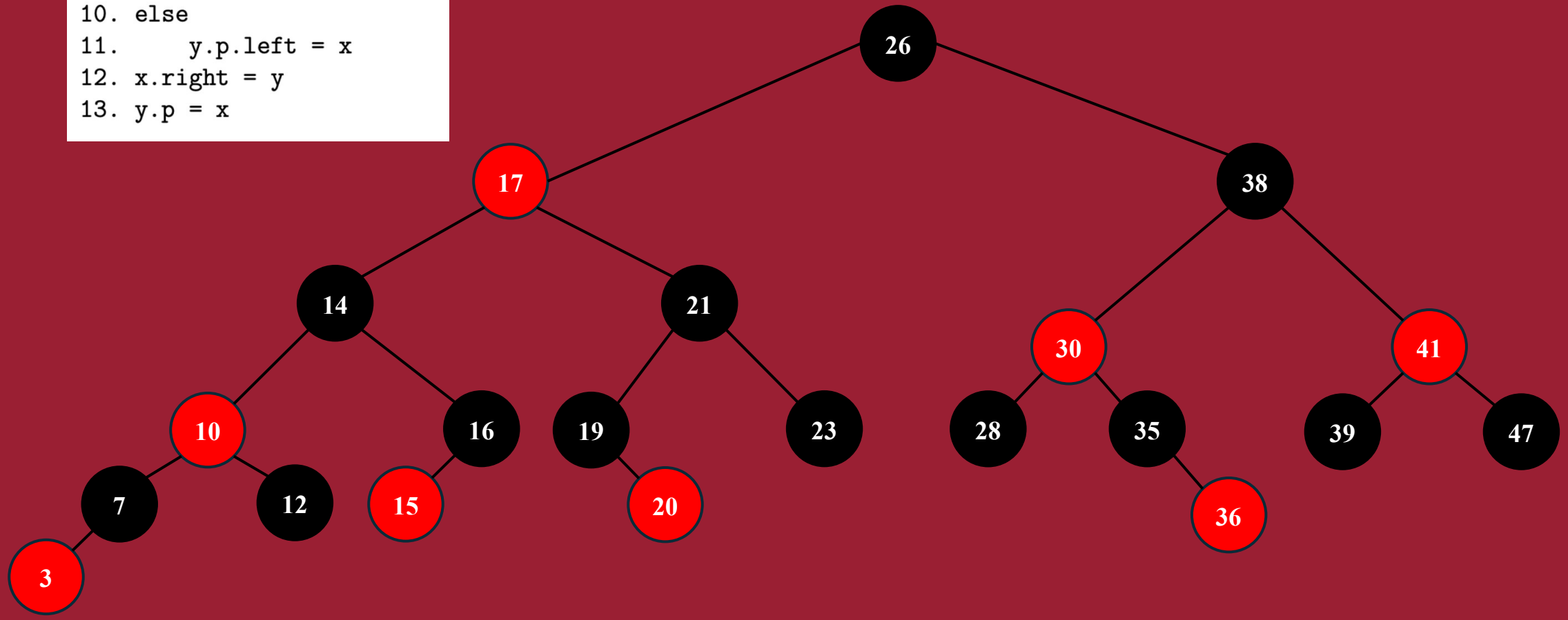
```
RIGHT-ROTATE(T, y)
x = y.left
y.left = x.right
if x.right != T.nil
    x.right.p = y
x.p = y.p
if y.p == T.nil
    T.root = x
else if y == y.p.right
    y.p.right = x
else
    y.p.left = x
x.right = y
y.p = x
```

…and *call RIGHT-ROTATE(T,z.p.p).*



```
RIGHT-ROTATE(T, y)
x = y.left
y.left = x.right
if x.right != T.nil
    x.right.p = y
x.p = y.p
if y.p == T.nil
    T.root = x
else if y == y.p.right
    y.p.right = x
else
    y.p.left = x
x.right = y
y.p = x
```

$x = y.left = 38;$
$y.left = x.right = 39;$
$x.right \neq T.nil,\ x.right.p = y = 38;$
$x.p = y.p = 26;$
$y == y.p.right,\ y.p.right = x = 38;$
$x.right = y = 41;$
$y.p = x = 38;$

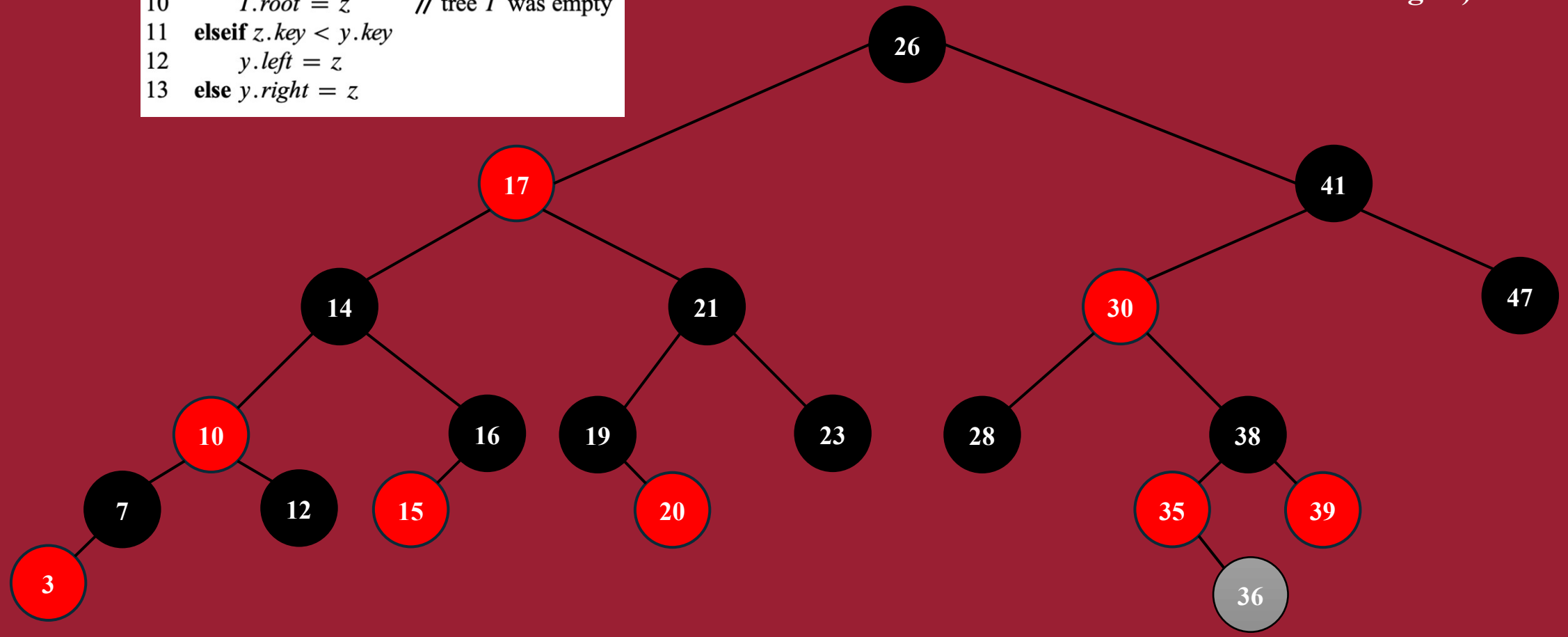


```
TREE-INSERT(T, z)
y = NIL
x = T.root
while x ≠ NIL
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
z.p = y
if y == NIL
    T.root = z          // tree T was empty
elseif z.key < y.key
    y.left = z
else y.right = z
```

TREE_INSERT does not assign *z* with a color. *Coloring 36 either red or black violates the Red-Black properties.*

Red breaks rule #4 (If a node is red, both of its children must be black.).

Black breaks rule #5 (For each node, all paths from the node to its descendant leaves must have the same black-height.).



Questions